

Different Ways of Using Variance Explained in Model Comparison

Michael S. Truong

York University

This document discusses different ways of using variance explained (R^2) as a means of model comparison. The primary aim is to reproduce and evaluate the model comparison procedure described in Hwang and Takane (2014) using the toy example of multiple linear regression (MLR). The secondary aim is to compare the replicated procedure against other potential approaches in terms of Type I error rates and statistical.

The direct quotes from Hwang & Takane (2014) are as follows:

“Similar to bootstrapping R-squared in linear regression (Ohtani 2000), we can compute the bootstrapped standard errors or confidence intervals of the difference in FIT between two models so as to examine whether there is a statistically significant difference in the FIT values. This procedure can be regarded as a nonparametric version of the paired t test for two groups of FIT values.”

— p.27

and

“Furthermore, we conducted a paired t-test for the FIT values between the original and simpler models. As described earlier, this test was equivalent to testing whether the difference in the FIT values calculated from 100 bootstrap samples for each model was equal to zero. We found the t-statistic nonsignificant statistically [$t(99) = -0.59$, $p = 0.55$, 95% $CI = -0.00-0.00$], indicating that there was no statistically significant difference in FIT between the original and simpler models.”

— p. 41

The above procedure is problematic because it is dependent on the number of bootstrap samples (B) and does not appropriately sample from the null distribution of no difference. As a

reminder, the formula for a one-sample t -test is¹

$$t = \frac{\bar{X} - \mu_0}{S/\sqrt{n}} \quad (1)$$

and is the same formula used for the paired t -test.² Specifically, the paired t -test compares the mean of the paired differences ($\bar{X} = \frac{x_{1n} - x_{2n}}{n}$) against μ_0 divided by the standard error of the paired differences $\frac{S}{\sqrt{n}}$. Additionally, in this context, $n = B$ where B is the number of bootstraps, therefore the (paired) t -statistic becomes arbitrarily large as you increase the number of bootstrap samples. This is a highly undesirable property of the procedure described by Hwang and Takane (2014). Also, the df (degrees of freedom) of the paired t -test is dependent on the number of bootstrap samples.

1. General Method

This is a two part simulation study. The first part uses the simple example of MLR in OLS to evaluate the original procedure by Hwang and Takane (2014) against other alternatives. The second part extends the simulation to the case of GSCA after a general literature review.

Claude Opus 4.8 helped suggest what conditions to compare for Study 1. Both simulations were coded with the assistance of Claude Opus 4.8. “All AI-assisted code was thoroughly reviewed, modified and independently verified by the author prior to execution.”³

¹Note that the paired t -test is equivalent to the formula for the one-sample t -test (). See Hogg et al. (2019, page 280), Wackerly et al. (2008, page 521, 646 to 648) and `getAnywhere('t.test.default')` for more information.

²I note the equivalence between the formulas because something that confused me was whether the paired t -test comparing the pairs of bootstrap differences in FIT statistic would be equivalent to the one-sample t -test of the differences against 0. They are equivalent.

³Direct quote of <http://arxiv.org/abs/2603.04003>

2. Study 1: Multiple Linear Regression

2.1. Method

2.1.1. Conditions

The fixed conditions are

```
1 sim_fixed <- list(  
2   beta0 = 0, beta1 = 0.3, beta2 = 0.3, sigma = 1, N = 200, alpha = 0.05,  
3   fit_compare = fit_compare  
4 )
```

which corresponds to Table 1.

Parameter	Value
β_0	0
β_1	0.3
β_2	0.3
σ	1
N	200
α	0.05

Table 1: Fixed parameters of the data-generating process.

The varying conditions are

```
1 SimDesign::createDesign(  
2   delta = c(0, 0.1, 0.2, 0.4),  
3   B      = c(100, 1000), # 100 reproduces H&T's df = 99  
4   prop1 = c(0.5, 0.7, 0.9) # fraction of N in part 1: 100/100, 140/60,  
5   180/20  
6 )
```

which corresponds to Table 2.

Factor	Levels
δ	0, 0.1, 0.2, 0.4
B	100, 1000
prop1	0.5, 0.7, 0.9

Table 2: Simulation design grid. $B = 100$ reproduces Hwang and Takane (2014)'s $df = 99$; prop1 is the fraction of N in part 1 (splits of 100/100, 140/60, 180/20).

2.1.2. Data Generating Process

A simple two-predictor data for a normally distributed response variable is generated. The coefficients β_1 and β_2 are incremented by δ based on whether case i is equal to group 2 or not.

```

1  Generate <- function(condition, fixed_objects) {
2    N      <- fixed_objects$N
3    # `prop1` = fraction of N in part 1 (0.5 = balanced). Clamp so both parts
      stay
4    # estimable (need n >= the 3 regression coefficients).
5    n1     <- min(N - 4L, max(4L, round(N * condition$prop1)))
6    group  <- factor(rep(c(1L, 2L), c(n1, N - n1)))
7    x1     <- rnorm(N)
8    x2     <- rnorm(N)
9    d2     <- as.integer(group == 2L)           # 1 in part 2,
      else 0
10   b1     <- fixed_objects$beta1 + condition$delta * d2      # slopes
      differ by delta
11   b2     <- fixed_objects$beta2 + condition$delta * d2
12   mu     <- fixed_objects$beta0 + b1 * x1 + b2 * x2
13   y      <- rnorm(N, mean = mu, sd = fixed_objects$sigma)
14   tibble::tibble(x1 = x1, x2 = x2, y = y, group = group)
15 }

```

2.1.3. Model Comparison Technique

```

1  # TODO: How does the stratification work? How do I know it's correct?
2  boot_mat <- boot::boot(dat, function(d, i) fit_compare(d[i, ]),
3                        R = B, strata = dat$group)$t

```

2.1.3.1. Hwang and Takane (2014) Procedure

```
1 # TODO: Do we want the two-sided or one-sided p-value? How do we get
  that? What does it mean regarding the sampling distribution?
2 p_ttest_R2 <- t.test(boot_mat[, "R2_split"], boot_mat[, "R2_pool"],
  paired = TRUE)$p.value
```

2.1.3.2. Wald Statistic Procedure

```
1 p_se_R2 <- 2 * pnorm(-abs(T_R2 / sd(dR2)))
2 p_se_adjR2 <- 2 * pnorm(-abs(T_adj / sd(dAdj)))
```

2.1.3.3. Percentile / ASL bootstrap CI of difference

```
1 # (3a) Percentile / ASL bootstrap CI of the difference.
2 p_pctl_R2 <- min(1, 2 * min(mean(dR2 <= 0), mean(dR2 >= 0)))
3 p_pctl_adjR2 <- min(1, 2 * min(mean(dAdj <= 0), mean(dAdj >= 0)))
```

2.1.3.4. Label Permutation Test

```
1 # TODO: How does the permutation work? How do I know it's correct?
2 perm_stat <- function(d, i) {
3   d$group <- d$group[i] # randomly reassign group labels
4   fc <- fit_compare(d)
5   c(R2 = unname(fc["R2_split"] - fc["R2_pool"]),
6     adj = unname(fc["adjR2_split"] - fc["adjR2_pool"]))
7 }
8 perm <- boot::boot(dat, perm_stat, R = B, sim = "permutation")
9 p_null_R2 <- (1 + sum(perm$t[, 1] >= perm$t0[1])) / (B + 1)
10 p_null_adjR2 <- (1 + sum(perm$t[, 2] >= perm$t0[2])) / (B + 1)
```

2.1.3.5. How R^2 was computed

For a single model, the R code is

```
1 SSE_pool <- sum(.lm.fit(X, y)$residuals^2)
```

```
2 SST <- sum((y - mean(y))^2)
3 R2_pool = 1 - SSE_pool / SST
```

This corresponds to the standard mathematical fomulation (James et al. 2021, Page 70, Eq. 3.17):

$$R_{\text{pooled}}^2 = 1 - \frac{SS_E}{SS_T}$$

$$= 1 - \frac{\sum_{i=1}^N (\hat{y}_i - y_i)^2}{\sum_{i=1}^N (y_i - \bar{y}_i)^2} \quad (2)$$

where $R_{\text{pooled}}^2 \in [0, 1]$.

For two separate models, the R code for R_{split}^2 is like so

```
1 SSE_split <- sum(.lm.fit(X[g1, , drop = FALSE], y[g1])$residuals^2) +
2               sum(.lm.fit(X[g2, , drop = FALSE], y[g2])$residuals^2)
3 SST <- sum((y - mean(y))^2)
4 R2_split = 1 - SSE_split / SST
```

This R_{split} approach is appropriate because it's analogous to what is done for GSCA in a multigroup model.⁴ Mathematically, this is equivalent to

$$R_{\text{split}}^2 = 1 - \frac{SS_{E1} + SS_{E2}}{SS_T}$$

$$= 1 - \frac{\sum_{i=1}^{n_1} (\hat{y}_i - y_i)^2 + \sum_{i=n_1+1}^N (\hat{y}_i - y_i)^2}{\sum_{i=1}^N (y_i - \bar{y}_i)^2} \quad (3)$$

where the data for model one ranges from case i to n_1 and the data for model two ranges from case $n_1 + 1$ to N .

We also compute adjusted- R^2 to better control for the number of additional parameters estimated when using split versus pooled models (James et al. 2021, Page 234, Eq. 6.4):⁵

⁴The aim of this study is to guide a simulation study for the context of GSCA, not to investigate what the optimal procedure for a OLS-only context is.

⁵The formula here computes $N - p$ as opposed to the $N - k - 1$ shown in James et al. (2021). The implicit context of James et al. (2021) is that the extra -1 term is needed to remove the intercept and that k —as it is used in James et al. (2021)—does not include the intercept. A proof of this is found in Section A.1.1.

$$\begin{aligned}\text{Adjusted } R^2_{\text{pooled}} &= 1 - \frac{\text{SS}_E / (N - p)}{\text{SS}_T / (N - 1)} \\ &= \frac{\sum_{i=1}^N (\hat{y}_i - y_i)^2 / (N - p)}{\sum_{i=1}^N (y_i - \bar{y}_i)^2 / (N - 1)}\end{aligned}\quad (4)$$

where p is the number of β -coefficients estimated in a model. In R, this can be done like so:

```
1 adjR2_pool = 1 - (SSE_pool / (n - p)) / (SST / (n - 1))
```

using the same SSE_pool and SST as computed above.

Mathematically, this adjusted- R^2 can be extended to the split model, like so:

$$\begin{aligned}\text{Adjusted } R^2_{\text{split}} &= 1 - \frac{(\text{SS}_{E1} + \text{SS}_{E2}) / (N - (p_1 + p_2))}{\text{SS}_T / (N - 1)} \\ &= 1 - \frac{\left(\sum_{i=1}^{n_1} (\hat{y}_i - y_i)^2 + \sum_{i=n_1+1}^N (\hat{y}_i - y_i)^2 \right) / (N - 2p)}{\sum_{i=1}^N (y_i - \bar{y}_i)^2 / (N - 1)}\end{aligned}\quad (5)$$

There are two assertions for proving Equation 5. The first is that for our specific case

$$p_1 + p_2 = 2p \quad (6)$$

where p is the number of parameters in the pooled model ($p = 3$). Equation 6 is proved by the fact that

```
1 y ~ group * (x1 + x2)
```

is equal to

```
1 y ~ 1 + group + x1 + x2 + group:x1 + group:x2
```

Listing 1: Expanded Interaction with $2p$ parameters

which has $2p$ parameters. The interaction model is relevant because Listing 1 is equivalent to fitting two separate `lm` models in R like so:⁶

⁶Due to dummy coding, the intercept in Listing 1 is the baseline of the dummy variable and group represents its contrast.

```
1 lm(y ~ 1 + x1 + x2, subset = group == 1)
2 lm(y ~ 1 + x1 + x2, subset = group == 2)
```

R

Listing 2: Two split `lm` models with 3 parameters each

Therefore, Equation 5 is the correct formula for the adjusted- R^2_{split} for our purposes.

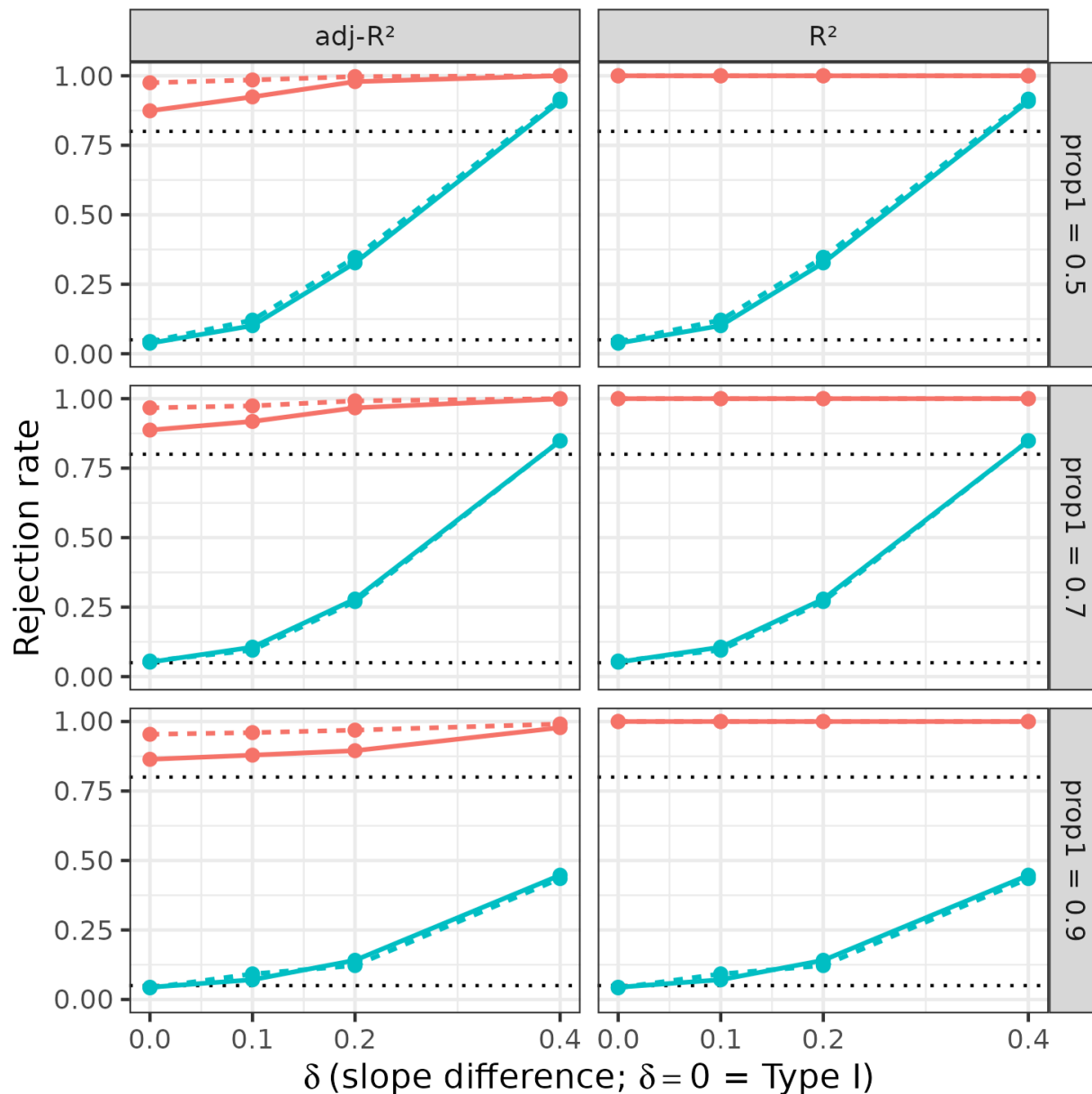
Importantly, the above proof shows how Equation 5 can be adapted to compute the adjusted FIT index for a multigroup GSCA type model. It shows how to sum the naive FIT values and it also shows why you can count the number of parameters, as if every parameter was interacting with ‘group’. Note that things might be more complex with constrained parameter estimates, but that is outside of the scope of my dissertation.

2.1.4. Outcome

Looking at Type I error rate and statistical power.

2.2. Results

Type I error ($\delta = 0$) and power ($\delta > 0$) by metho



Method — H&T — Permutation Bootstraps — B = 100 - - - B = 1000
Dotted line = nominal α

Figure 1: Rejection rates over 1000 Monte Carlo replications at $\alpha = 0.05$, as a function of the between-group slope difference δ . The leftmost point of each curve ($\delta = 0$) is the empirical **Type I error rate**; points at $\delta > 0$ give **power**. Columns vary the group-size imbalance (prop1, the fraction of N in part 1: balanced 100/100 to severe 180/20); rows separate raw R^2 from adjusted R^2 . Line type distinguishes the number of bootstrap resamples B . The dotted line marks the nominal α . The Hwang and Takane (2014) paired t -test (red) rejects almost always even at $\delta = 0$ — on raw R^2 its Type I error is 1.0 at every condition, and on adjusted R^2 it rises with B (dotted above solid), exposing the $\sqrt{B}$ artifact. The label-permutation test (green) and Wald statistic (blue) stay

The same rejection rates are tabulated in Table 3.

Method	B	R^2				adj- R^2			
		0	0.1	0.2	0.4	0	0.1	0.2	0.4
$n_1/n_2 = 100/100$									
H&T	100	1.000	1.000	1.000	1.000	0.874	0.924	0.979	1.000
H&T	1000	1.000	1.000	1.000	1.000	0.975	0.985	0.997	1.000
Permutation	100	0.038	0.101	0.327	0.908	0.038	0.101	0.327	0.908
Permutation	1000	0.044	0.121	0.347	0.916	0.044	0.121	0.347	0.916
$n_1/n_2 = 140/60$									
H&T	100	1.000	1.000	1.000	1.000	0.887	0.918	0.967	0.999
H&T	1000	1.000	1.000	1.000	1.000	0.967	0.974	0.992	1.000
Permutation	100	0.052	0.106	0.279	0.848	0.052	0.106	0.279	0.848
Permutation	1000	0.056	0.095	0.270	0.849	0.056	0.095	0.270	0.849
$n_1/n_2 = 180/20$									
H&T	100	1.000	1.000	1.000	1.000	0.864	0.879	0.895	0.978
H&T	1000	1.000	1.000	1.000	1.000	0.954	0.960	0.969	0.991
Permutation	100	0.044	0.071	0.141	0.447	0.044	0.071	0.141	0.447
Permutation	1000	0.042	0.092	0.122	0.435	0.042	0.092	0.122	0.435

Note. Columns within each metric block index the slope difference δ . Cells are the proportion of replications rejecting H_0 at $\alpha = .05$; $\delta = 0$ is the Type I error rate and $\delta > 0$ is power. Row groups give the part-1/part-2 sample sizes; the deviating subgroup is part 2.

Table 3: Empirical rejection rates over 1000 Monte Carlo replications at $\alpha = .05$, by model-comparison method and number of bootstrap resamples B . The $\delta = 0$ columns give the Type I error rate; $\delta > 0$ columns give power.

Ignore:

1. pct1 method
 - Over reject
2. Chow method
 - Seems to make assumptions about the parameters that aren't made in GSCA. So won't pursue further.

2.3. Discussion

3. Study 2: Generalized Structured Component Analysis

3.1. Method

3.2. Results

3.3. Discussion

4. General Conclusion

A Appendix

A.1 Proofs

A.1.1 Adjusted R^2 Formula

Here is a LLM-assisted numerical proof of why the -1 term is needed for James et al. (2021)'s version of the adjusted R^2 formula, but not for ours in Equation 5.

```
1  fit <- lm(mpg ~ hp + wt, data = mtcars)
2  y   <- mtcars$mpg
3  X   <- model.matrix(fit)           # [1, hp, wt]
4  n   <- length(y)
5  p   <- ncol(X)
6  SSE <- sum(residuals(fit)^2)
7  SST <- sum((y - mean(y))^2)
8
9  lm_adj <- summary(fit)$adj.r.squared
10 manual_np <- 1 - (SSE/(n - p)) / (SST/(n - 1)) # our formula
11 manual_np1 <- 1 - (SSE/(n - p - 1)) / (SST/(n - 1)) # the "extra -1"
   variant
12
13 paste0("n = ", n, ", p = ncol(X) = ", p)
14 paste0("lm() adj.r.squared      : ", round(lm_adj, 3))
15 paste0("manual 1-(SSE/(n-p))    : ", round(manual_np, 3), " identical?
   ", isTRUE(all.equal(lm_adj, manual_np)))
16 paste0("manual 1-(SSE/(n-p-1)) : ", round(manual_np1, 3), " identical?
   ", isTRUE(all.equal(lm_adj, manual_np1)))
```

Listing 3: R proof of altered formula.

which returns

```
1 "n = 32, p = ncol(X) = 3"
2 "lm() adj.r.squared      : 0.815"
3 "manual 1-(SSE/(n-p))    : 0.815 identical? TRUE"
4 "manual 1-(SSE/(n-p-1)) : 0.808 identical? FALSE"
```